

PHP grunder

Börja här! Förutsättningar för att följa PHP-skolan

Information om PHP

Information om Apache webserver

Installation av Apache ->

Installation av PHP ->

Grunderna i PHP

Kommentera din kod

Variabler

Konstanter

Villkorssatser (kontrollstrukturer)

Operatorer

Slingor (loopar)

Vektorer (arrays)

nästa steg : PHP fortsättning ->

Börja här! Förutsättningar för att följa PHP-skolan

PHP-skolan börjar från grunden med installation av de program du behöver och genomgång av funktionerna. Vi utgår ifrån att du inte har några kunskaper alls i PHP men om du har det är det naturligtvis en fördel. Däremot bör du ha grundläggande kunskaper i HTML för att ha utbyte av informationen i guiderna. PHP och databaser är stora områden och vi rekommenderar att du avsätter en hel del tid för att gå igenom guiderna i PHP-skolan. Att bara läsa exemplen utan att prova dem själv kommer inte att ge dig kunskaperna att själv konfigurera och använda mer komplexa PHP-script på din webbplats. Att utveckla PHP-skolan har tagit mycket lång tid för oss och vi hoppas att det även tar en hel del tid för dig att följa guiderna :-)

PHP-skolan är uppdelad i fler delar än de du ser i vår huvudmeny. Om du följer den här guiden avsnitt för avsnitt så följer du den väg vi har tänkt oss när vi byggt guiderna. Om du däremot vill hoppa mellan guiderna bör du vara medveten om att de praktiska exemplen som används följer med i alla guiderna. Utförliga förklaringar finns där funktionerna visas första gången och upprepas inte varje gång funktionen återkommer. Vi rekommenderar att du följer guiderna i den här ordningen:

1. [Installera Apache i Windows »](#)

2. [Installera PHP »](#)

3. [PHP grunder »](#)

4. [PHP fortsättning »](#)

5. [Installera MySQL »](#) 

6. [MySQL och databaser »](#)

7. [PHP och MySQL »](#)

De guider du hittar här nu är en grund både för dig och oss och vår målsättning är att nya guider och tips med konkreta exempel på hur du kan använda PHP och MySQL ska tillkomma i PHP-skolan!

En värdefull resurs för att få hjälp och tips är att besöka [WDS Forum »](#) där du kan besöka PHP-forumet.

Information om PHP

PHP är en förkortning av "PHP Hypertext Preprocessor" där själva förkortningen ingår som första ord i förkortningen vilket kallas för återkommande akronym (recursive acronym). Namnet "Hypertext Preprocessor" antyder också vad PHP gör - förbehandlar kod som sedan visas som hypertext i webbläsaren. PHP är ett scriptspråk som skrivs direkt i HTML-koden där instruktionerna utförs av webservern och inte i webbläsaren. Om du väljer att visa källkoden i webbläsaren så ser du inte PHP-koden då den redan är interpreterad till HTML av webservern innan den skickas till webbläsaren (klienten). PHP har likheter med CGI-script (Perl) men då CGI-script körs som egen fil så infogas PHP direkt i HTML-koden. PHP används för att skapa dynamiska och interaktiva sidor som e-butiker, forum, gästböcker, communities, formulär, inloggningssystem mm. PHP kan alltså användas tillsammans med databaser där information kan hämtas och sparas. Du har även stor användning av PHP i dina "vanliga" HTML-dokument. Ett exempel är menyer som återanvänds på flera sidor i din webbplats - genom att infoga samma menysida i alla dokument med PHP-kod så kan du ändra menyn i ett enda dokument och menyn på alla sidor uppdateras - du har då dynamiska sidor med hjälp av PHP. Du kan alltså inte använda endast PHP för att bygga dina webplatser utan som ett komplement till HTML.

Webdesignskolan har valt PHP framför ASP, ASP.net, ColdFusion och andra serverside-språk då PHP är gratis och utvecklas opensource. Dessutom kan PHP köras på flera olika webbservrar och vi visar här installation av PHP tillsammans med Apache webserver som liksom PHP är gratis och opensource. ASP är utvecklat av Microsoft och kräver deras egen webserver IIS (Internet information server) eller PWS (Personal webserver).

Läs mer om installationen av PHP i guiden ["installera PHP" »](#)

Information om Apache webserver

Programmet Apache används på mer än 70% av alla webbservrar vilket är mer än alla andra webserverprogram tillsammans. Om du använder ett webhotell och framförallt om du har ett UNIX-konto är chansen stor att du redan idag använder Apache webserver - läs mer om webhotell i guiden [Webhotell och domännamn »](#). OBS! Microsoft's webbservrar IIS och PWS kan idag inte installeras under Windows XP Home och du måste ha Windows XP Professional, Windows 98, Windows 95, Windows NT eller Windows 2000. Apache webserver kan du däremot installera på Windows XP Home och de flesta andra versioner av Windows och även UNIX. Läs mer om [Apache Software Foundation »](#)

OBS! Du behöver inte installera PHP eller Apache för att använda PHP! Det är bara om du vill testa dina sidor lokalt på din egen dator innan du publicerar dem till din webserver som du behöver installera PHP och Apache. Om ditt webhotell har PHP installerat kan du använda PHP direkt och du ser då resultatet när du publicerat dokumentet till webservern. Om du vill testa dina sidor lokalt eller vill använda din dator som en webserver måste du däremot installera både PHP och Apache (eller annan webserver).

Läs mer om installationen av Apache webserver i guiden ["installera Apache i Windows" »](#)

Grunderna i PHP

PHP-dokumentet bearbetas av en webserver och har filändelsen **.php** (och inte .htm som dina vanliga HTML-sidor). Du kan alltså inte prova exemplen nedan och öppna dokumentet direkt i webbläsaren som då inte kan läsa PHP-koden. Publicera dina dokument till en webserver eller webhotell med **PHP-stöd** eller **installera** PHP och Apache på din egen dator så kan du testa dina PHP-dokument lokalt.

Läs mer om installationen av PHP i guiden ["Installera PHP" »](#)

Förutsättningar för att följa guiden

OBS! Förutsättningar för att följa guiden: alla exempel som visas i denna guide använder **PHP installerat lokalt** och Apache webserver bearbetar koden innan den visas i webbläsaren.

Den lokala rotmappen är **C:\www** om du installerat Apache och PHP enligt guiden ["installera PHP" »](#) och adressen till din lokala webserver är då **http://localhost** eller **http://127.0.0.1**

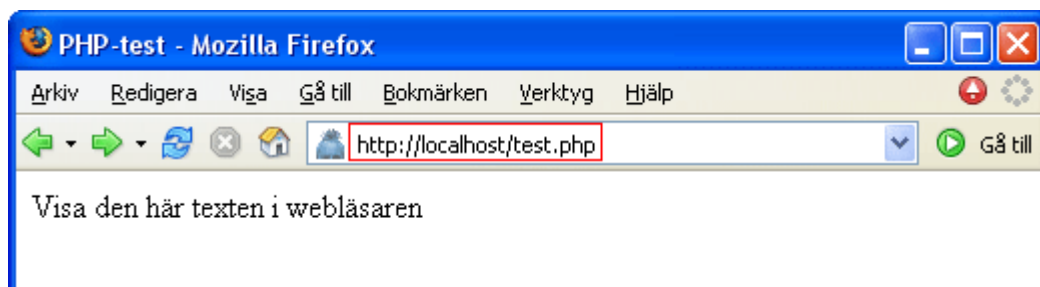
Skriva ett PHP-dokument med avgränsare för PHP

Då PHP körs inbäddad i HTML-kod måste avgränsare användas för att webservern ska tolka vad som är PHP-kod. Den avgränsare som används är **<?php** och **?>** (du kan även använda **<?** och **?>** men det rekommenderas inte då det kan misstolkas om du använder XML-dokument).

1. Skriv koden nedan och spara filen med namnet **test.php** i din rootmapp **C:\www**

```
<html>
<head>
<title>PHP-test</title>
</head>
<body>
<?php
echo "Visa den här texten i webbläsaren";
?>
</body>
</html>
```

2. Öppna dokumentet i din webbläsare genom att ange **http://localhost/test.php** i adressfältet:



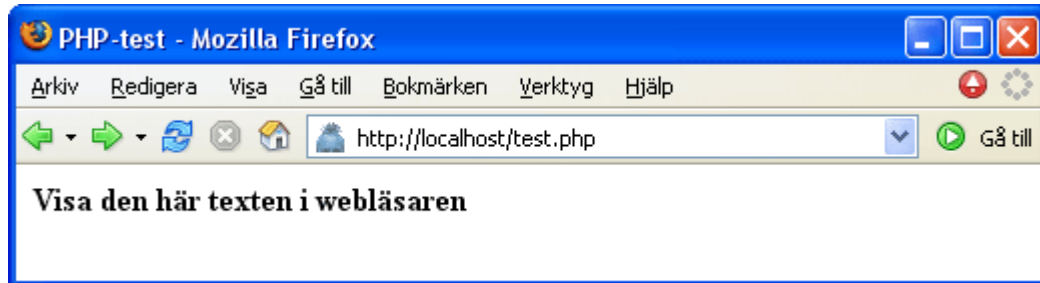
OBS! När du gör en ändring i koden sparar du dokumentet och **uppdaterar** sedan webbläsaren med **F5** för att se resultatet.

3. Formatering med vanlig HTML kan du ange inom PHP-koden:

```
<html>
<head>
<title>PHP-test</title>
</head>
```

```
<body>
<?php
echo "<strong>Visa den här texten i webbläsaren</strong>";
?>
</body>
</html>
```

Här visas texten formaterad med fetstil :



Du får samma resultat i exemplet om du anger formateringen omslutande PHP-koden:

```
<html>
<head>
<title>PHP-test</title>
</head>
<body>
<strong>
<?php
echo "Visa den här texten i webbläsaren";
?>
</strong>
</body>
</html>
```

Kommentera din kod

Att ange egna kommentarer och anteckningar i koden är viktig både för dig själv och för andra som ska läsa koden och få en bra bild över vad den utför. Kommentering är information som inte ska tolkas av PHP utan bara vara just information om vad koden utför. Avgränsare för kommentarer är `//` eller `#` för enradig kommentar och `/*` för flerradig kommentar.

Skriv in dina egna kommentarer på en och flera rader och testa om de är osynliga i webbläsaren:

```
<html>
<head>
<title>PHP-test</title>
</head>
<body>
<?php
// kommentar på en rad
# kommentar på en rad
/* Kommentar som sträcker sig över flera rader och passar bra för omfattande information om
koden. Det här är mitt första PHP-dokument och det fungerar bra hittills! */
echo "Visa den här texten i webbläsaren";
?>
</body>
```

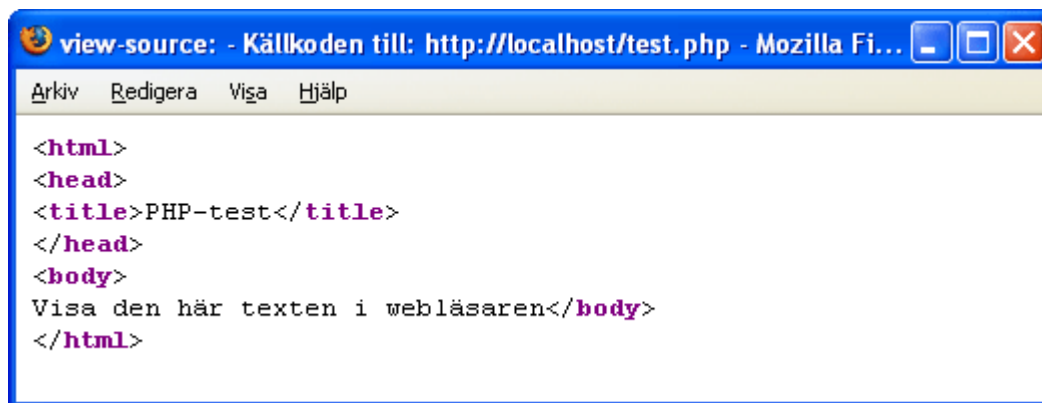
```
</html>
```

Observera att flerradiga kommentarer inleds med `/*` och avslutas med `*/`

Källkoden i webbläsaren

Om du väljer att visa koden för dokumentet med menyn "Visa/Källkod" (Visa/Källa) så ser du inte PHP-koden utan bara resultatet av PHP-koden tillsammans med den vanliga HTML-koden. Nu ser du alltså vad som beskrivits i inledningen till denna guide; PHP förbehandlar kod som sedan visas som hypertext i webbläsaren. PHP är ett scriptspråk som skrivs direkt i HTML-koden där instruktionerna utförs av webservern och inte i webbläsaren. Om du väljer att visa källkoden i webbläsaren så ser du inte PHP-koden då den redan är interpreterad till HTML av webservern innan den skickas till webbläsaren (klienten).

Här är resultatet av exemplet ovan med kommentering av koden (som inte heller syns):



Variabler

En variabel är information som finns sparad och kan hämtas när den behövs. Motsatsen till variabel är en konstant. Informationen i variabeln kan finnas i samma PHP-dokument eller hämtas i annat dokument eller en databasfil. Variabeln är alltså en typ av behållare för varierande information och kan vara av olika typer som tex: strängar (string) heltal (integer) flyttal, decimaltal (floating point, double) vektorer (array). Strängar kan innehålla både text och siffror och variabeln omsluts då med citationstecken " (dubbelfnuttar).

En variabel inleds med ett dollartecken **\$** och måste börja med en bokstav som tex **\$fnamn** och variabelnamnet är "case sensitive" dvs skiljer på versaler/gemener. Detta innebär att **\$Fnamn** inte är samma namn och detta är då två olika variabelnamn trots samma stavning. Variabelnamn får inte börja med en siffra och namnet **\$1fnamn** är alltså felaktigt men kan då istället inledas med "understrykningsstreck" (underscore) som tex **\$_1fnamn** för att bli ett giltigt variabelnamn.

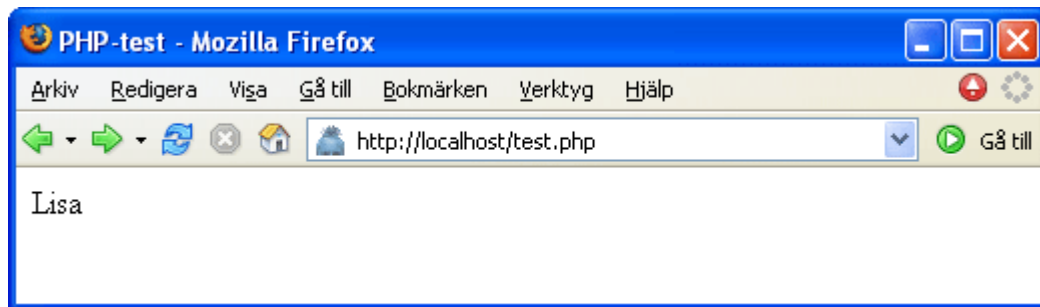
Följande exempel på hur variabler används kan verka poänglösa då variabeln anges i samma kodavsnitt men tänk dig istället att en variabel kan ändras av användaren genom att skriva namnet i ett formulär eller hämtas ur en databas när vissa villkor uppfylls.

1. Skriv ditt eget förnamn som variabel och låt förnamnet hämtas upp och visas i webbläsaren:

```
<html>
<head>
<title>PHP-test</title>
```

```
</head>
<body>
<?php
$fnamn = "Lisa";
echo "$fnamn";
?>
</body>
</html>
```

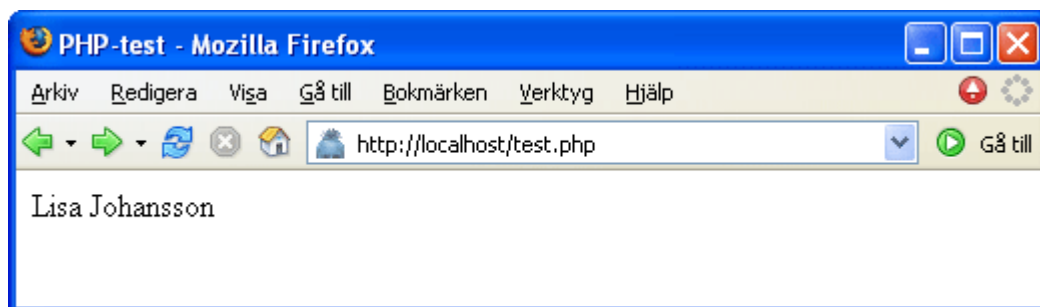
En variabel är en behållare för information som kan hämtas in. Informationen som i exemplet är ett förnamn hämtas in och visas:



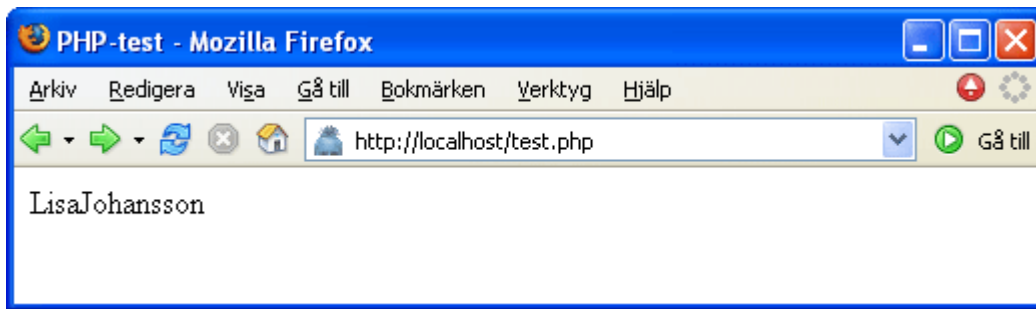
2. Du kan kombinera strängvariabler genom att använda punkttecknet. Skriv in ditt efternamn som en ny variabel och hämta både förnamnet och efternamnet i samma sträng genom att binda samman dem med punkt:

```
<html>
<head>
<title>PHP-test</title>
</head>
<body>
<?php
$fnamn="Lisa";
$enamn = "Johansson";
echo "$fnamn" . " $enamn";
?>
</body>
</html>
```

Notera mellanslaget i " \$enamn" som ger resultatet:



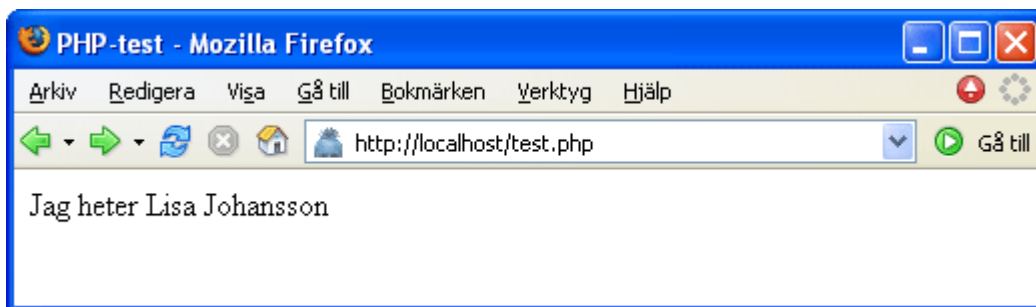
Om du inte anger mellanslag i "\$enamn" får du resultatet:



3. Du kan också kombinera vanlig **text** och strängvariabler genom att använda **punkttecknet** för att binda ihop dem.

```
<?php
$fnamn = "Lisa";
$enamn = "Johansson";
echo "Jag heter $fnamn" . " $enamn";
?>
```

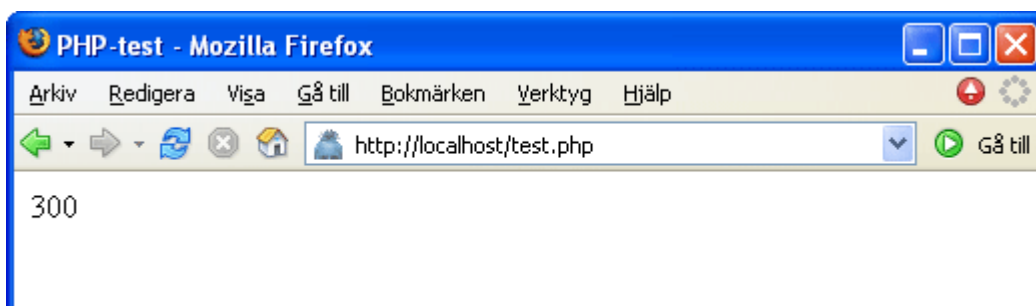
Här är vanlig **text** kombinerad med information som hämtats i **variabler**:



4. För summering av två tal använder du **plustecknet**. Talen är angivna som variabler men observera att inga citationstecken (dubbelfnuttar) används runt siffrorna. Inte heller behöver du använda fnuttarna i strängen som används för att visa summan:

```
<?php
$ta1 = 100;
$ta2 = 200;
echo $ta1+$ta2;
?>
```

De två variablerna \$ta1 och \$ta2 innehåller talen 100 och 200 och summan blir då:

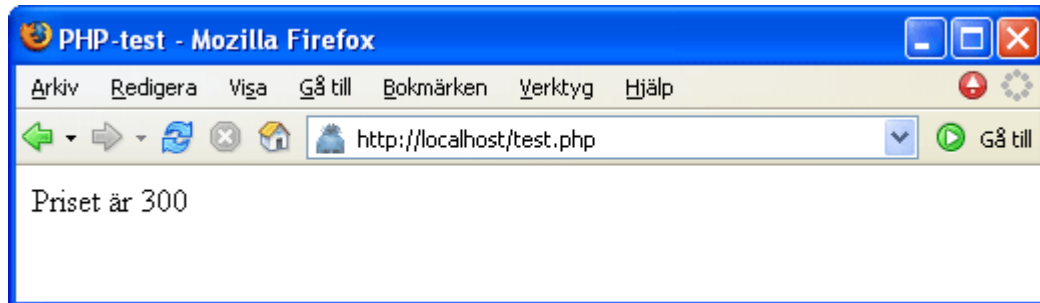


5. Kombinera tal och text genom att omsluta texten med citationstecken (men inte talen) och bind ihop strängvariabeln och summeringen av de två heltalsvariabeln med kommatecken:

```
<?php
```

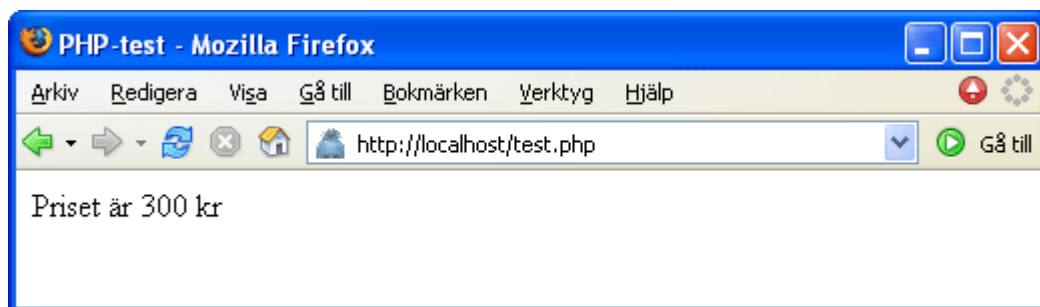
```
$tal1 = 100;  
$tal2 = 200;  
echo "Priset är " , $tal1+$tal2;  
?>
```

Vanlig text som är kombinerad med heltalsvariabel:



6. Om du vill lägga till en **enhet** efter heltalsvariabeln binder du samman den med en punkt:

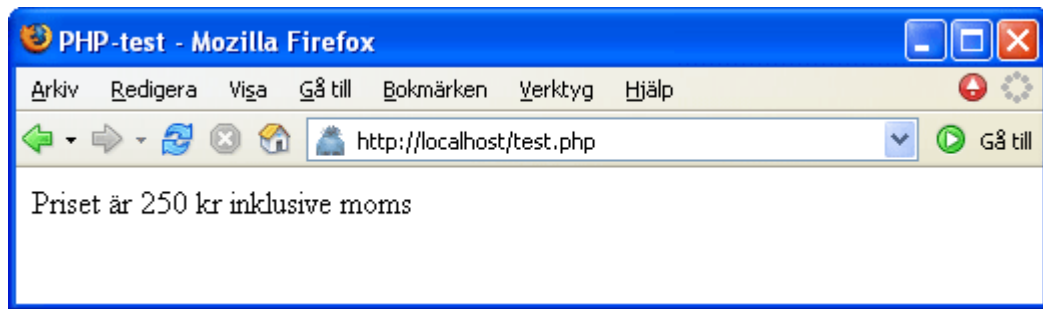
```
<?php  
$tal1 = 100;  
$tal2 = 200;  
echo "Priset är " , $tal1+$tal2 . " kr";  
?>
```



7. De fyra räknesätten kallas i PHP för aritmetiska operatorer (läs mer om operatorer längre ned) och exemplet nedan visar hur du kan använda multiplikation i en decimaltalsvariabel för att räkna ut moms på priset. Observera att punkt används istället för kommatecken i decimaltalen:

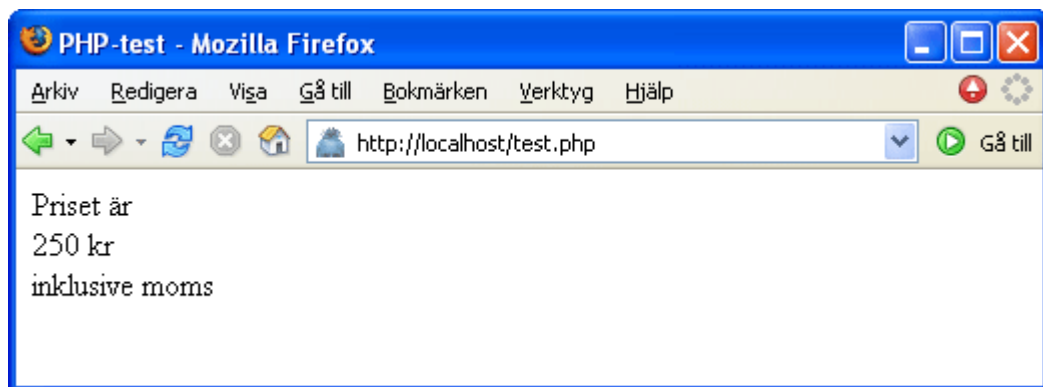
```
<?php  
$pris = 200;  
$moms = 1.25;  
$totalpris = $pris * $moms;  
echo "Priset är " . $totalpris . " kr inklusive moms";  
?>
```

I exemplet är `$totalpris` den nya variabeln och uträkningen (multiplikationen) är redan gjord i variabeln och du kan då använda punkter för att binda samman text och talvariabel:



8. **Radbrytning** i text och strängar gör du med `<br>` eller `<br />` och enklast är då att lägga den inom strängen omsluten av citationstecknen:

```
<?php
$pris = 200;
$moms = 1.25;
$totalpris = $pris * $moms;
echo "Priset är <br /> " , $totalpris , " kr <br /> inklusive moms";
?>
```



Superglobala variabler

PHP tillhandahåller ett stort antal **fördefinierade** variabler med namn som alltså inte kan användas till egna variabler. Många av de sk "superglobala" variablerna är beroende av vilken webserver och konfiguration som används och PHP anger inte alla superglobala variabler som kan användas. Läs mer om de fördefinierade variablerna (superglobals) hos [PHP.net](http://php.net) »

Här är några av de vanligaste superglobala variablerna:

`$_SERVER` `$_ENV` `$_COOKIE` `$_GET` `$_POST` `$_FILES` `$_REQUEST` `$_SESSION`

TIPS! Superglobalen **`$_POST`** används i avsnittet om ["Formulär"](#) » , superglobalen **`$_SESSION`** används i avsnittet ["Sessioner"](#) » och superglobalen **`$_COOKIE`** används in avsnittet ["Cookies"](#) »

Konstanter

En variabel är information som finns sparad och kan hämtas när den behövs. Motsatsen till variabel är en konstant som alltså inte ändras när scriptet körs. En konstant måste definieras innan den används och du kan själv bestämma namnet på konstanten. Till skillnad från variabler inleds inte konstanten med dollartecken \$ och en regel (men inget krav) är att använda versaler i namnet för att lättare skilja

den från variabler i koden. Konstantnamnet är liksom variabelnamnet "case sensitive" dvs skiljer på versaler/gemener.

Exempel på definiering av en **konstant**:

```
define ("COMPANY", "Webdesignskolan");
```

som anger att företagsnamnet är konstant och gäller under hela programkörningen.

Om företagsnamnet istället ska kunna bytas ut används **variabeln**:

```
$company="Webdesignskolan";
```

där företagsnamnet kan bytas ut under programmets gång.

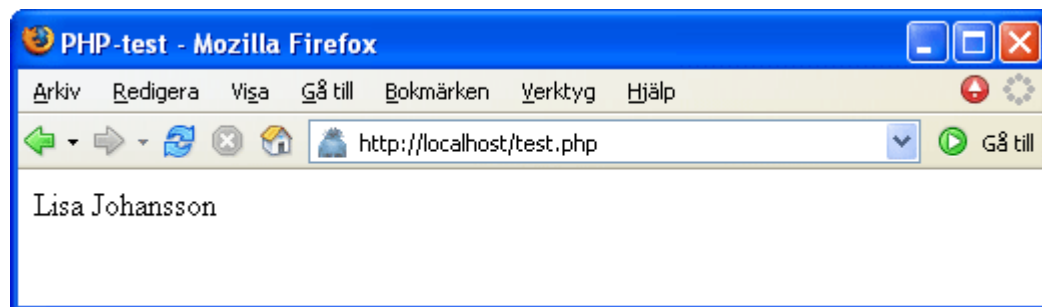
1. Definiera **konstanterna** och visa dem med "echo". I exemplet används dessutom **echo " ";** för att skapa ett blanksteg mellan för- och efternamn:

```
<?php
define ("FNAMN", "Lisa");
define ("ENAMN", "Johansson");
echo FNAMN;
echo " ";
echo ENAMN;
?>
```

Konstanterna i exemplet ovan utför samma sak som **variablerna** i exemplet nedan:

```
<?php
$fnamn = "Lisa";
$enamn = "Johansson";
echo "$fnamn";
echo " ";
echo "$enamn";
?>
```

Skillnaden är att innehållet i variablerna kan bytas under programmets gång medan konstanterna inte kan ändras. Användning av konstanter eller variabler enligt exemplen ovan ger samma resultat i webbläsaren:



Villkorssatser (kontrollstrukturer)

Ett PHP-script bygger på olika uttryck som tex kan vara en uppgift, funktionsanrop eller villkorsformulering. Ett uttryck avslutas med ett semikolon och kan grupperas i **block** omslutna med måsvingar { krullparenteser }.

En av de viktigaste villkorssatserna är **if** som tillsammans med **else** och **elseif** används för att

kontrollera om ett villkor är uppfyllt eller ej. Villkoren fungerar som "sant eller falskt" och om villkoret inte är sant (if) är det alltså falskt (else).

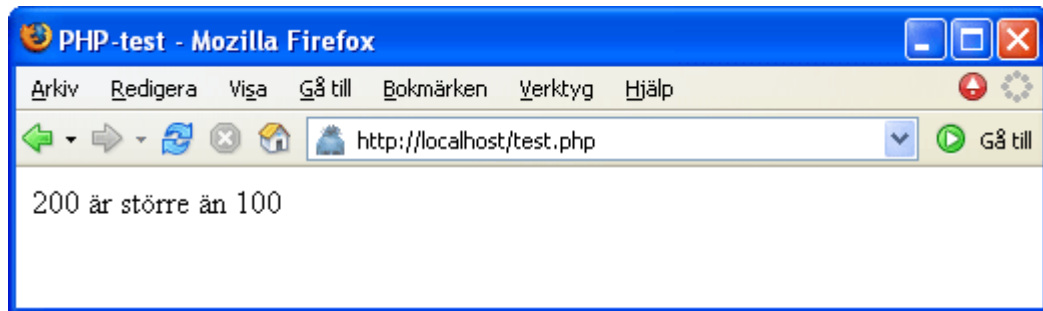
1. Exemplet nedan jämför två tal och returnerar ett meddelande beroende på vilket tal som angivits (vilket villkor som är uppfyllt):

```
<?php
$ta1 = 200;
$ta2 = 100;

if ($ta1 > $ta2)
{
    echo "$ta1 är större än $ta2";
}

else
{
    echo "$ta1 är mindre än $ta2";
}
?>
```

ta1 är större än ta2 och villkoret **if** är uppfyllt och då anges meddelandet:



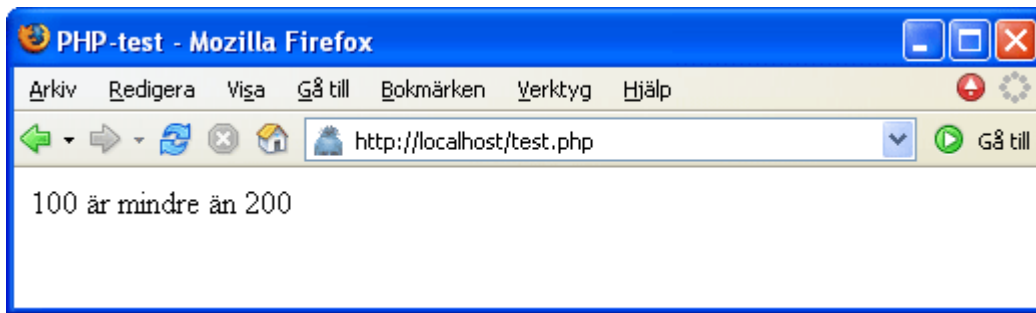
2. Samma exempel men nu är ta1 lägre än ta2:

```
<?php
$ta1 = 100;
$ta2 = 200;

if ($ta1 > $ta2)
{
    echo "$ta1 är större än $ta2";
}

else
{
    echo "$ta1 är mindre än $ta2";
}
?>
```

ta1 är mindre än ta2 och villkoret **if** är inte uppfyllt och istället anges meddelandet för **else**:



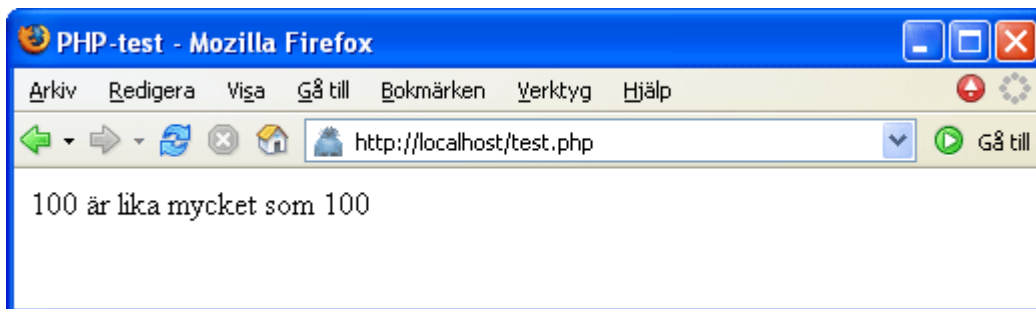
3. Du kan bygga på med fler villkor genom att använda **elseif** (eller om). Om det första villkoret (if) är falskt kontrolleras nästa villkor (elseif) och utförs om det är sant (om varken if eller elseif är sanna återstår else).

I exemplet nedan kontrolleras om tal1 är större än tal2 **eller om** tal1 är mindre än tal2. Om inget av villkoren är uppfyllda återstår bara att det är falskt (else):

```
<?php
$ta1 = 100;
$ta2 = 100;

if ($ta1 > $ta2) {
    echo "$ta1 är större än $ta2";
}
elseif ($ta1 < $ta2) {
    echo "$ta1 är mindre än $ta2";
}
else {
    echo "$ta1 är lika mycket som $ta2";
}
?>
```

I koden ovan är tal1 varken är större eller mindre än tal2 och då återstår alltså villkoret falskt (else):



4. Villkorssatsen **elseif** kan anges flera gånger för att ange flera olika villkor som ska kontrolleras och när ett av villkoren stämmer utförs detta. I exemplet nedan anges 4 olika prisnivåer på en vara och villkoren är att priset **\$pris** ska vara **mindre än** en angiven prisnivå. Du som kund ger olika kommentarer beroende på vad du anser om priset:

```
<?php
$pris = 49;

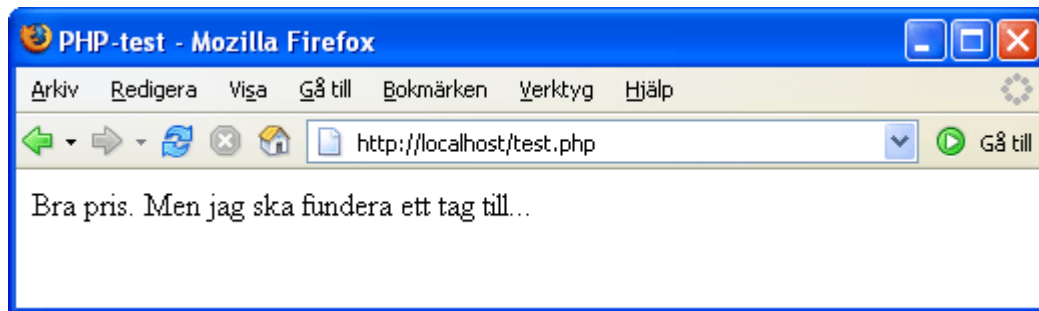
if ($pris < 10) {
    echo "Vilket fynd! Jag köper direkt!";
}
elseif ($pris < 20) {
    echo "Ja, det är både billigt och prisvärt";
}
elseif ($pris < 50) {
    echo "Bra pris. Men jag ska fundera ett tag till...";
}
```

```

}
elseif ($pris < 100) {
    echo "Hmm.. låter dyrt";
}
else {
    echo "Glöm det! Jag letar vidare i en annan affär";
}
?>

```

Priset som angivits ovan (49 kr) är tydligen lite för dyrt för att du ska slå till och köpa varan direkt:



5. Ett annat exempel på hur **elseif** kan användas där siffran 1 till 12 visar motsvarande månad:

```

<?php

$month = 1;
if ($month == 1) {echo "Januari";}
elseif ($month == 2) {echo "Februari";}
elseif ($month == 3) {echo "Mars";}
elseif ($month == 4) {echo "April";}
elseif ($month == 5) {echo "Maj";}
elseif ($month == 6) {echo "Juni";}
elseif ($month == 7) {echo "Juli";}
elseif ($month == 8) {echo "Augusti";}
elseif ($month == 9) {echo "September";}
elseif ($month == 10) {echo "Oktober";}
elseif ($month == 11) {echo "November";}
elseif ($month == 12) {echo "December";}
else {echo "Du har angivit ett felaktigt värde för månadens nummer!";}
?>

```

(Notera att här används den jämförande operatoren **==** som inte ska förväxlas med tilldelningsoperatoren **=** vilket är ett vanligt misstag och dessutom kan vara svårt att felsöka i koden.)

6. Du kan använda villkorssatsen **switch** som ett komplement till if-satsen för att kontrollera om ett villkor är uppfyllt eller ej. Villkoret **switch** är detsamma som att ange en serie av **else if**-satser för samma uttryck och värdet i uttrycket anges som ett argument, **case**. Exemplet som visas här är mycket förenklat och fördelen mot att använda koden ovan kan vara svår att överblicka. En stor fördel är att i ett switch-uttryck utvärderas villkoret endast en gång och resultatet jämförs med varje argument (case). I ett else if-uttryck utvärderas villkoret igen och om ditt villkor är mer komplicerat eller i en loop kan en switch vara snabbare.

Villkoret **\$month** utvärderas här endast en gång men med istället med flera argument:

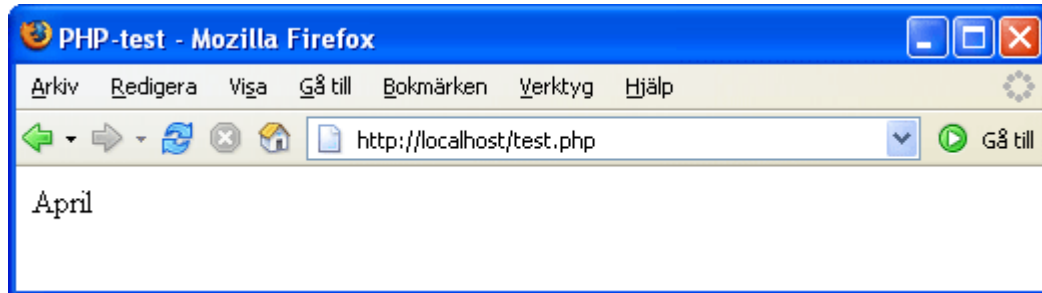
```

<?php
$month = 4;
switch ($month)
{
    case 1: echo "Januari"; break;
    case 2: echo "Februari"; break;
    case 3: echo "Mars"; break;
}

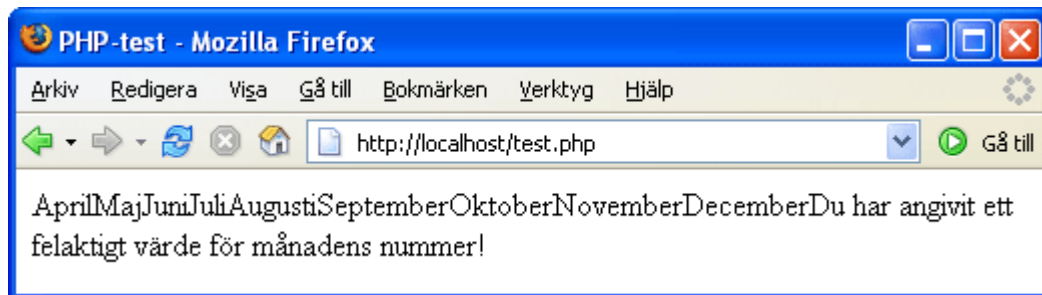
```

```
case 4: echo "April"; break;
case 5: echo "Maj"; break;
case 6: echo "Juni"; break;
case 7: echo "Juli"; break;
case 8: echo "Augusti"; break;
case 9: echo "September"; break;
case 10: echo "Oktober"; break;
case 11: echo "November"; break;
case 12: echo "December"; break;
default: echo "Du har angivit ett felaktigt värde för månadens nummer!"; break;
}
?>
```

Koden ovan med månadssiffran 4 ger resultatet "April":



Villkoret **switch** anges inom ett enda **block** omslutna med **{** krullparenteser **}** och PHP kör igenom alla argument till slutet av blocket oavsett om de uppfyller det angivna villkoret eller inte. Genom att ange **break** stoppas körningen när rätt argument är uppfyllt. Om du inte anger **break** någonstans skulle resultatet i vårt exempel se ut så här:



Operatorer

I de villkorssatser du använt ovan har du redan använt operatorer för att jämföra värden i de angivna variablerna. När du utför en vanlig addition använder du operatoren **+** (plustecken) för att lägga samman två eller flera tal: $100+200=300$ som också kan skrivas med med variabler: $\$a+\$b=\$c$

De finns olika typer av operatorer och här visas ett urval utan några detaljerade förklaringar eller exempel. Använd detta avsnitt som en referens till andra delar av PHP-guiden där det visas praktiska exempel och förklaringar om hur du använder operatorerna. Läs mer om operatorer på [PHP.net](http://php.net) »

Aritmetiska operatorer

- + Addition
- Subtraktion
- * Multiplikation

/	Division	
%	Modulus	Resten av divisionen mellan två tal
Jämförande operatorer		
==	lika med	
===	identiska	Är lika med och dessutom av samma typ
!=	inte lika med	
<>	inte lika med	Mer eller mindre men inte lika med
!==	inte identiska	Är inte lika och inte av samma typ
<	mindre än	
>	större än	
<=	mindre än eller lika med	
>=	större än eller lika med	
Logiska operatorer		
and	och	
or	eller	
xor	exklusivt eller	Om något av alternativen stämmer men inte båda
!	inte	
&&	och	Högre prioritet skiljer "&&" från "and" när det gäller företrädesrätten (precedence) läs mer på PHP.net »
	eller	Högre prioritet skiljer " " från "or" när det gäller företrädesrätten (precedence) läs mer på PHP.net »

Slingor

Slingor (**loopar**) används för att upprepa något flera gånger utan att behöva skriva koden om igen. Att skriva ut eller behandla i tur och ordning kallas också för att **iterera** över någonting. Ett exempel är att iterera över en tabell och skriva ut innehållet radvis (läs mer om att iterera över en tabell i avsnittet "Vektorer" nedan).

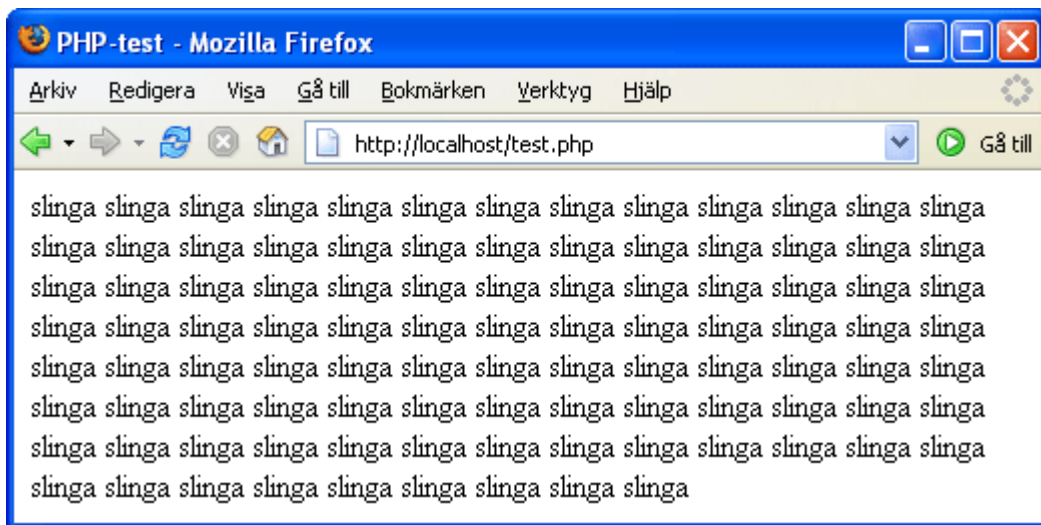
Det finns tre huvudtyper av slingor; **while** - **for** - **foreach**

Slingan **while** upprepar (loopar) körningen av koden i ett block så länge ett angivet villkor är uppfyllt. Men däremot behöver du inte veta innan hur många gånger det kan behöva köras utan det körs så länge ett villkor är uppfyllt. Detta innebär att while är användbart när du vill hämta poster ur en databas där du inte vet innan hur många poster som matchar dina sökvillkor.

1. I exemplet nedan upprepas ordet "slinga" ända tills det uppnår det angivna villkoret 100 gånger:

```
<?php
$i = 1;
while ($i <= 100)
{
    $i++;
    echo "slinga ";
}
?>
```

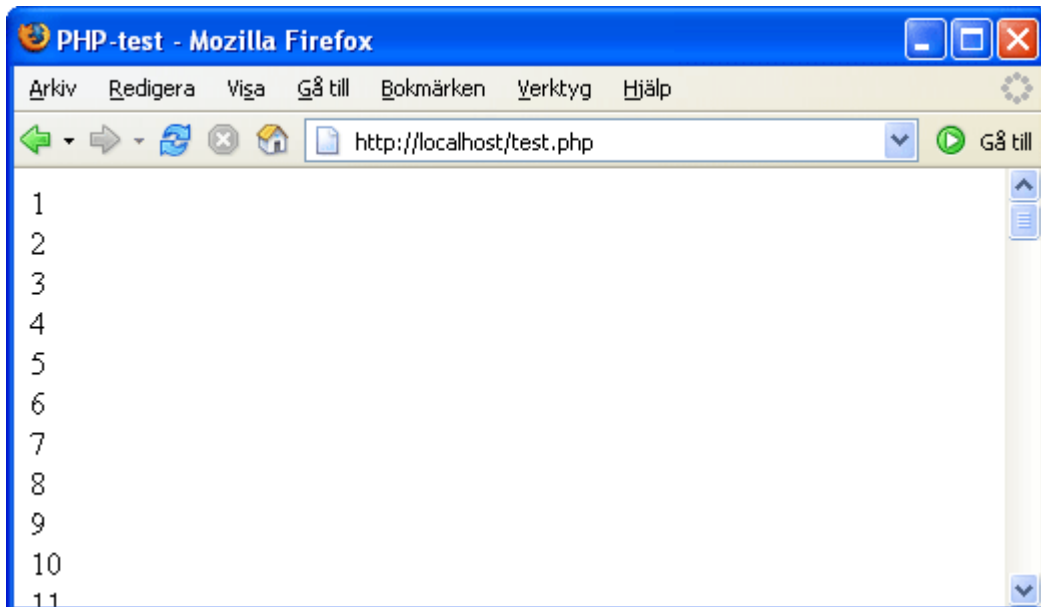
Ordet "slinga" upprepas 100 gånger:



2. Ett till exempel på användning av **while** är uppräknningen av tal tills angivet villkor uppfylls. Här används dessutom `<br />` för att ange varje tal på en egen rad (notera också att **punkt** används för att binda samman variabel och text):

```
<?php
$i = 1;
while ($i <= 100)
{
    echo $i++ . "<br />";
}
?>
```

Talet ökar på varje ny rad upp till 100 rader:



Slingan **for** förutsätter att du vet hur många gånger den ska köras och syntaxen är:

for (expr1; expr2; expr3) statement

Det första uttrycket (expr1) utvärderas och körs villkorslöst. Det andra uttrycket (expr2) utvärderas också och om det är sant (TRUE) fortsätter slingan att köras och framställningen (statement) utförs. Om det andra uttrycket (expr2) däremot är falskt (FALSE) stoppas körningen av slingan. I slutet av varje iteration utvärderas och körs det tredje uttrycket (expr3). De tre delarna i for-slingan kan också beskrivas så här:

for (initiering; villkor; förändring) framställning

1. Om vi använder samma exempel som med "while" ovan så ser koden ut så här:

```
<?php
for ($i=1; $i <= 100; $i++)
{
    echo $i . "<br />";
}
?>
```

Det första uttrycket **\$i=1** körs alltid och nästa uttryck **\$i <= 100** endast om villkoret är sant (TRUE). Det tredje uttrycket **\$i++** anger att uppräknings ska ske vilket också utförs 100 gånger. Framställningen är angiven till utskrift av resultatet (**echo \$i . "
"**).

Skillnaden mellan **while** och **for** i exemplen ovan är endast att koden blir uppbyggd på ett prydligare sätt men i mer avancerad användning kan du ha större nytta av att kunna välja mellan att använda while eller for.

Slingan **foreach** itererar över vektorer och du kan läsa mer om det i nästa avsnitt "Vektorer".

Vektorer (arrays)

En **vektor** (**array**) är en variabel som kan lagra mycket information och istället för att skapa en mängd variabler för att hantera stora mängder data är det enklare att samla dem i en vektor. En vektor innehåller element som består av **nycklar** och **värden**. Nyckeln (kallas också index) används för att ange vilket värde som ska bearbetas. Observera att den första heltalsnyckeln börjar med noll (0) om inget annat anges. Du kan också tilldela nycklarna egna namn istället för siffror.

1. Det första exemplet visar en vektor med **automatisk indexering** av fyra element som innehåller nycklarna 0, 1, 2, 3 och ett värde (medlemsnamn) för varje nyckel. Inom hakparenteserna [] behöver alltså inget nyckelvärde anges men den automatiska indexeringen börjar alltid med heltalsnyckeln 0 (noll). När vi vill skriva ut namnet (värdet) på medlem 1 så är det alltså inte det första värdet i vektorn (Lisa) utan det andra värdet (Anna):

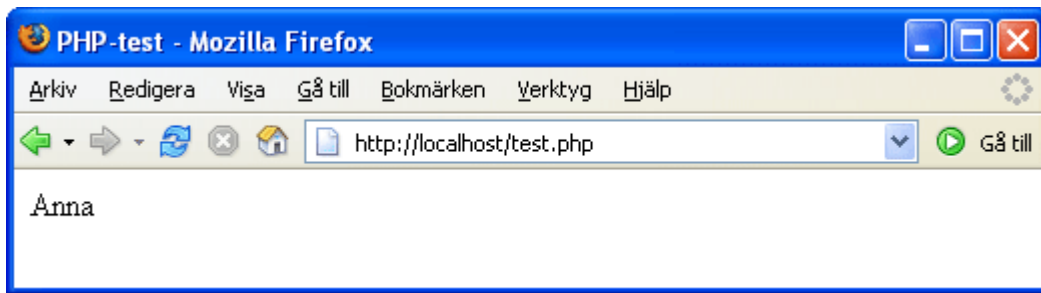
```
<?php

$medlem [] = "Lisa";
$medlem [] = "Anna";
$medlem [] = "Karin";
$medlem [] = "Märta";

echo $medlem[1];

?>
```

Medlemsnamnet (värdet) som visas är det **andra** i ordningen (som har nyckeln 1):



2. Om du vill visa det första elementets värde (medlem "Lisa") så anger du **0** (noll) som nyckel:

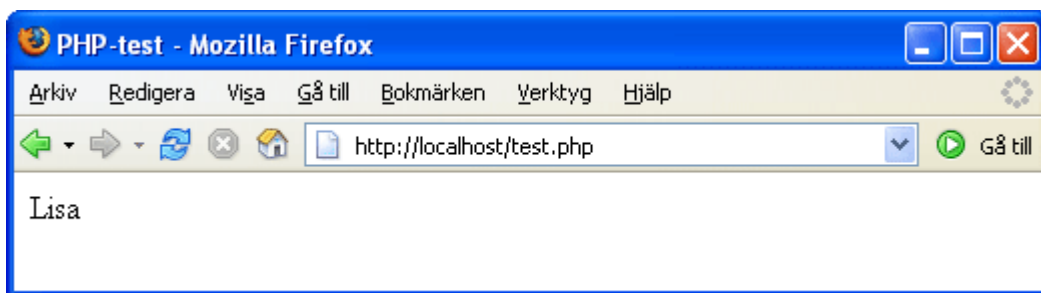
```
<?php

$medlem [ ] = "Lisa";
$medlem [ ] = "Anna";
$medlem [ ] = "Karin";
$medlem [ ] = "Märta";

echo $medlem[0];

?>
```

Lisa är det **första** namnet (värdet) i medlemslistan och har nyckeln **0**:



3. Om du vill bestämma nycklarna själv anger du detta inom hakparentesen och vektorn får då en **manuell indexering**. Koden nedan visar medlem (värdet) "Lisa" som har tilldelats nyckeln 1 :

```
<?php

$medlem [1] = "Lisa";
$medlem [2] = "Anna";
$medlem [3] = "Karin";
$medlem [4] = "Märta";

echo $medlem[1];

?>
```

4. Du behöver inte ange en nyckel till alla element utan det räcker att du anger den nyckel indexeringen ska **starta** med:

```
<?php

$medlem [1] = "Lisa";
$medlem [ ] = "Anna";
$medlem [ ] = "Karin";
$medlem [ ] = "Märta";

echo $medlem[1];
```

```
?>
```

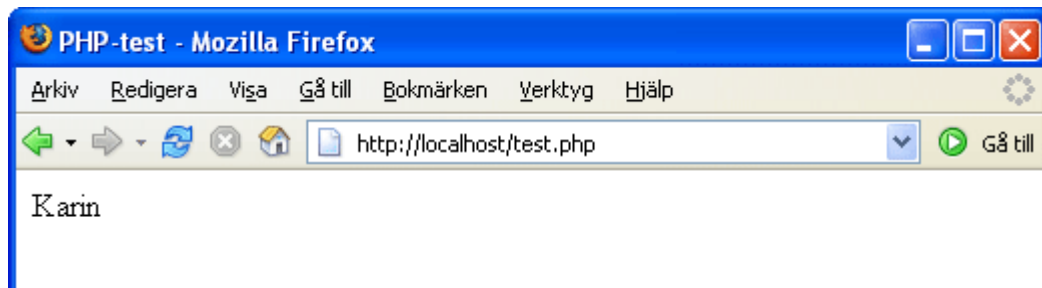
5. Ett annat sätt att ordna alla element i en vektor är att använda språkkonstruktionen **array ()** som ger samma resultat som den tidigare medlemslistan men elementet **\$medlem** behöver då inte upprepas för varje medlem (värde) i vektorn:

```
<?php  
  
$medlem = array("Lisa", "Anna", "Karin", "Märta");  
  
echo $medlem[3];  
  
?>
```

6. Exemplet ovan visar medlem "Märta" som tilldelats nyckeln 3 automatiskt. Om du vill ange manuella nycklar kan du även här ange den nyckel indexeringen ska **starta** med. Här anges att **1 =>** ska vara startnyckel och de övriga nycklarna räknas då upp automatiskt:

```
<?php  
  
$medlem = array(1 => "Lisa", "Anna", "Karin", "Märta");  
  
echo $medlem[3];  
  
?>
```

Nu är det medlem (värdet) "Karin" som har tilldelats nyckeln 3:

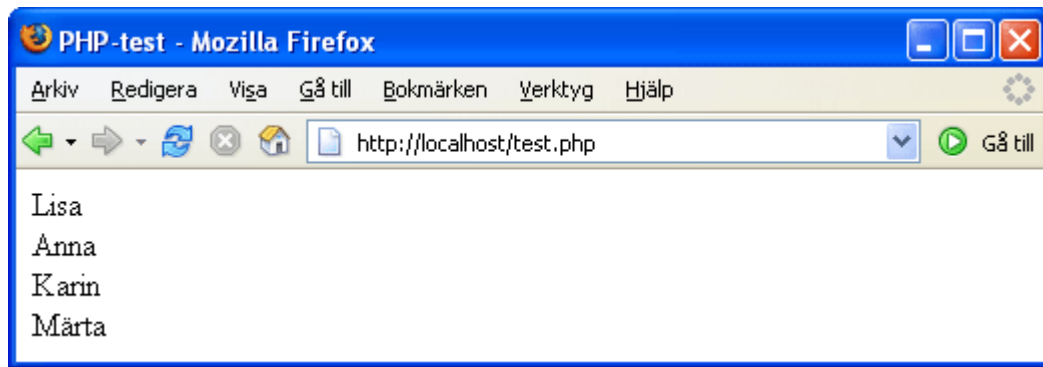


Slingan **foreach** (som vi nämnde sist i avsnittet om "Slingor" ovan) är användbar i vektorer (arrays) då den loopar igenom en bit kod för varje element i vektorn. Till skillnad från slingan **for** behöver du inte veta i förväg hur många gånger den ska köras (loopa) utan den itererar över alla element i vektorn.

1. Här används foreach i vektorn för att hämta alla värden och skriva ut dem med radbrytning mellan varje medlem. Variabeln **\$värde** skapas för att hämta värden i varje element i vektorn och anges med **\$medlem as \$varde** (men variabeln kan egentligen ha vilket namn du vill, tycker du att "namn" är mer passande för värdet i dina nycklar kan du ange **\$medlem as \$namn** istället).

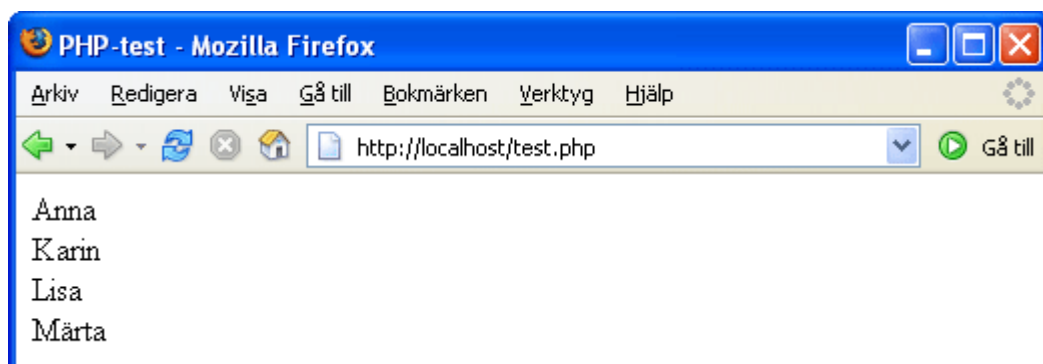
```
<?php  
  
$medlem = array("Lisa", "Anna", "Karin", "Märta");  
  
foreach ($medlem as $varde)  
  
echo $varde . "<br />";  
  
?>
```

Resultatet av slingan **foreach** är en lista med alla medlemsnamn (värden):



2. **Sortera** din vektor (innan den skrivs ut) i bokstavsordning med funktionen **sort** :

```
<?php  
  
$medlem = array("Lisa", "Anna", "Karin", "Märta");  
sort($medlem);  
  
foreach ($medlem as $varde)  
  
echo $varde . "<br />";  
  
?>
```

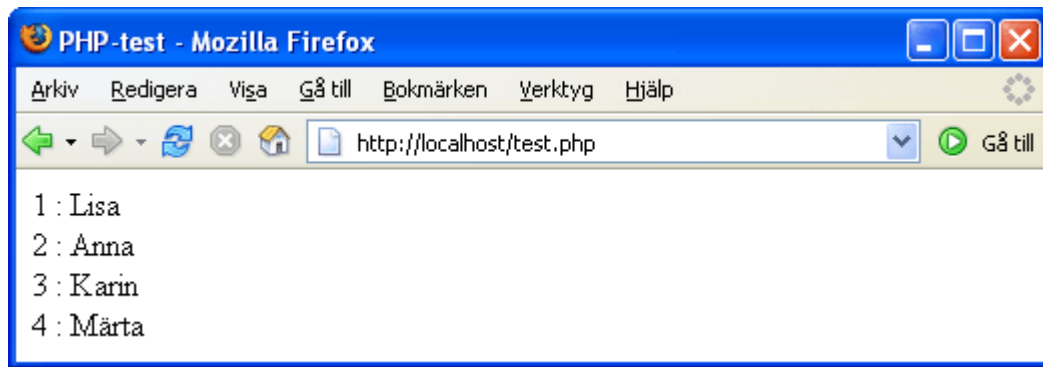


TIPS! Använd **arsort** istället om du vill sortera din vektor i omvänd bokstavsordning:
`arsort($medlem);`

3. Om du vill använda foreach för att hämta både nycklar och värden och vill indexera manuellt med egna värden så anger du dina **nyckelvärden** i vektorn (för att undvika automatiskt indexering med start på nyckeln 0). Exemplet nedan visar hur ett medlemsnummer angivits som nyckel före varje namn (värde) i vektorn. Variabelnamnen som används här är **\$nyckel** och **\$varde** men de motsvarar alltså "medlemsnr" och "namn":

```
<?php  
  
$medlem = array(1 => "Lisa", 2=> "Anna", 3=> "Karin", 4=> "Märta");  
  
foreach ($medlem as $nyckel => $varde)  
  
echo $nyckel . " : " . $varde . "<br />";  
  
?>
```

Resultatet är att varje medlemsnamn (värde) visas med medlemsnummer (nyckel):



En vektor kan innehålla andra vektorer och kallas då **flerdimensionell vektor**. I vårt exempel med medlemslistan kan vi placera in medlemmarna i olika grupper. Vi skapar grupper med namnen "Grupp1", "Grupp2" och "Grupp3" och anger vilka personer som ska ingå i varje grupp. Namnen vi använt tidigare i vår medlemslista är fortfarande i samma vektor men ingår nu i vektorn Grupp1, med andra ord en vektor i en vektor (array i en array):

```
$medlem = array("Grupp1" => array("Lisa", "Anna", "Karin", "Märta")
```

1. Vi bygger nu ut medlemslistan med två grupper till som också får innehålla fyra personer vardera. För att komma åt och skriva ut innehållet i alla vektorerna krävs **två foreach-slingor**. Den första foreach-slingan hämtar nyckeln för gruppnamnen som vi kallat just för **\$nyckel**. När nästa slinga påbörjas används värdet **\$varde** nu som en vektor:

```
<?php

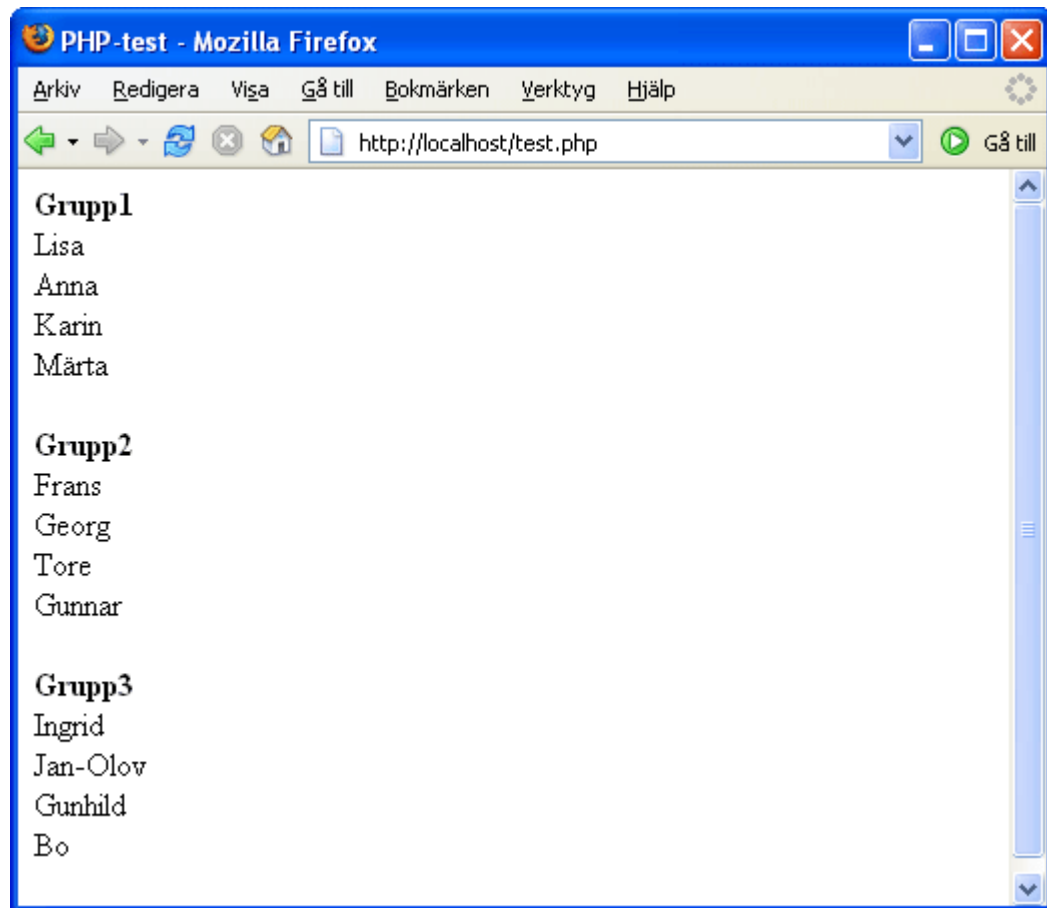
$medlem = array(
    "Grupp1" => array("Lisa", "Anna", "Karin", "Märta"),
    "Grupp2" => array("Frans", "Georg", "Tore", "Gunnar"),
    "Grupp3" => array("Ingrid", "Jan-Olov", "Gunhild", "Bo")
);

foreach ($medlem as $nyckel => $varde) {
    echo "<b>" . $nyckel . "</b><br />";

    foreach ($varde as $varde2) {
        echo $varde2 . "<br />";
    }
    echo "<br />";
}

?>
```

Resultatet ger en gruppindelad lista:



TIPS! Arrays är mycket användbara och finns som flera funktioner - läs mer på [PHP.net](http://php.net) »

Nu har du grundkunskaperna i PHP och kan gå vidare till guiden [PHP fortsättning](#) »

[Copyright © **Webdesignskolan**
Materialet får skrivas ut och användas för personligt bruk.
Användning i undervisningssyfte är ej tillåten utan vårt tillstånd - [läs mer här](#) »]